

```
<view
    class="com.android.notepad.NoteEditor$MyEditText"
    id="@+id/note"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="@android:drawable/empty"
    android:padding="10dip"
    android:scrollbars="vertical"
    android:fadingEdge="vertical" />
```

- o Thành phần tùy chỉnh được tạo dưới dạng một view chung trong XML, còn lớp được xác định bởi gói đầy đủ. Lưu ý, lớp nội bộ chúng ta vừa định nghĩa được tham chiếu bằng cách sử dụng ghi chú (notation) `NoteEditor$MyEditText`, đây là cách chuẩn để tham chiếu tới các lớp nội bộ trong ngôn ngữ lập trình Java.

Nếu thành phần View tùy chỉnh không được định nghĩa dưới dạng một lớp nội bộ thì cách thay thế là bạn có thể khai báo thuộc tính View với tên phần tử XML và loại bỏ thuộc tính class. Ví dụ:

```
<com.android.notepad.MyEditText
    id="@+id/note"
    ... />
```

Lưu ý, lớp `MyEditText` lúc này nằm trong một file riêng biệt. Khi lớp đó được lồng trong lớp `NoteEditor`, kỹ thuật này sẽ không hoạt động.

- o Các thuộc tính và tham số khác trong định nghĩa được truyền vào phương thức khởi tạo thành phần tùy chỉnh, sau đó truyền qua phương thức khởi tạo `EditText`, nên chúng là những tham số giống nhau mà bạn có thể sử dụng cho một view `EditText`. Lưu ý, bạn cũng có thể tự thêm các tham số của mình, chúng ta sẽ bàn lại việc này sau.

Và đó là tất cả những gì bạn cần làm. Phải thừa nhận rằng đây là một trường hợp đơn giản, song lại là chìa khóa để tạo ra các thành phần tùy chỉnh chỉ vừa đủ phức tạp ở mức bạn cần.

Một thành phần phức tạp hơn có thể ghi đề lên các phương thức `on...` nhiều hơn, đồng thời đưa ra một số phương thức trợ giúp của chúng, về căn bản là để tùy chỉnh các thuộc tính và hành vi. Giới hạn duy nhất là trí tưởng tượng của bạn và những gì bạn muốn thành phần đó thực hiện.

6. Xử lý đầu vào từ bàn phím

Hệ thống Android đưa ra một bàn phím trên màn hình (on-screen keyboard) - được biết đến như một phương tiện nhập liệu mềm (soft input method) - khi một trường văn bản trong giao diện người dùng được focus. Để cung cấp trải nghiệm tốt nhất cho người dùng, bạn có thể xác định các đặc điểm về loại đầu vào mình muốn (chẳng hạn như nó là số điện thoại hay địa chỉ e-mail) và hành vi của phương tiện nhập liệu (chẳng hạn như nó có thực hiện hành vi tự sửa lỗi về ngữ pháp hay không).

Ngoài các phương tiện nhập liệu trên màn hình, Android còn hỗ trợ bàn phím cứng, nên điều quan trọng là ứng dụng của bạn phải tối ưu trải nghiệm người dùng để tương tác có thể xảy ra, cho dù đó là bàn phím gắn kèm.

Những chủ đề này được bàn luận ở các mục dưới đây.

6.1 Xác định kiểu phương tiện nhập liệu

Mỗi trường văn bản đều phục vụ cho một loại văn bản đầu vào tương ứng, chẳng hạn như địa chỉ e-mail, số điện thoại, hay chỉ đơn thuần là văn bản dạng chữ. Vì vậy, điều quan trọng là bạn phải xác định kiểu đầu vào cho từng trường văn bản trong ứng dụng của mình, nhằm giúp hệ thống hiển thị phương tiện nhập liệu mềm phù hợp (chẳng hạn như một bàn phím trên màn hình).

Bên cạnh những loại button có sẵn kèm theo phương tiện nhập liệu, bạn nên xác định các hành vi, chẳng hạn như phương tiện này có cung cấp gợi ý chính tả (spelling suggestion) không, hoặc viết hoa đầu dòng các câu mới và thay thế button ngắt dòng bằng một button action, chẳng hạn như Done hoặc Next. Mục này sẽ chỉ cho bạn thấy cách xác định những đặc điểm trên.

Xác định kiểu bàn phím

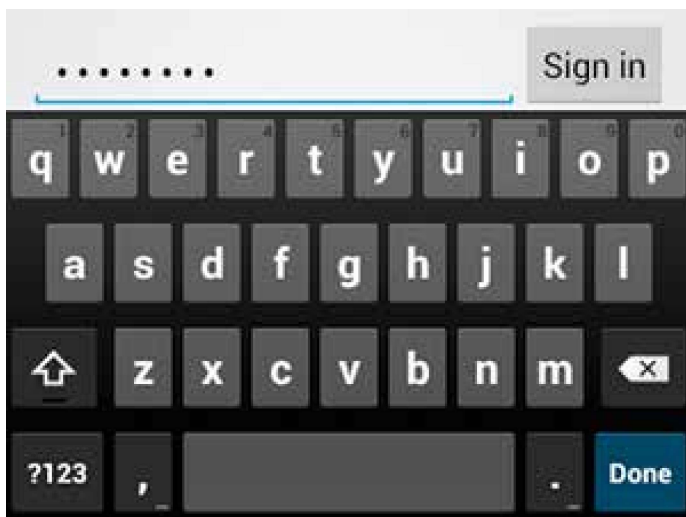
Bạn hãy luôn khai báo phương tiện nhập liệu cho trường văn bản của mình bằng cách thêm thuộc tính `android:inputType` vào phần tử `<EditText>`.



Hình 1. Kiểu đầu vào phone.

Ví dụ, nếu bạn muốn có một phương tiện nhập liệu cho việc nhập số điện thoại, hãy sử dụng giá trị "phone":

```
<EditText
    android:id="@+id/phone"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:hint="@string/phone_hint"
    android:inputType="phone" />
```



Hình 2. Kiểu đầu vào `textPassword`.

Hoặc, nếu trường văn bản là dành cho mật khẩu, hãy sử dụng giá trị `"textPassword"` để trường đó che (conceal) dữ liệu đầu vào khi người dùng nhập:

```
<EditText
    android:id="@+id/password"
    android:hint="@string/password_hint"
    android:inputType="textPassword"
    ... />
```

Một số giá trị có thể được ghi lại với thuộc tính `android:inputType` và một số giá trị có thể được kết hợp để định nghĩa giao diện cho phương tiện nhập liệu cùng những hành vi thêm vào.

Bật gợi ý chính tả và các hành vi khác



Hình 3. Thêm `textAutoCorrect` sẽ cung cấp khả năng tự sửa các lỗi chính tả.

Thuộc tính `android:inputType` cho phép bạn xác định nhiều hành vi cho phương tiện nhập liệu. Quan trọng nhất, nếu trường văn bản của bạn được dành cho đầu vào dạng văn bản cơ bản (chẳng hạn như cho một tin nhắn văn bản), bạn nên bật tính năng tự sửa chính tả với giá trị `"textAutoCorrect"`.

Bạn có thể kết hợp các hành vi và kiểu phương tiện nhập liệu khác nhau với thuộc tính `android:inputType`. Ví dụ, đây là cách để tạo một trường văn bản có chức năng viết hoa từ đầu tiên của một câu và tự sửa lỗi chính tả:

```
<EditText
    android:id="@+id/message"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:inputType=
        "textCapSentences|textAutoCorrect"
.../>
```

Xác định action của phương tiện nhập liệu

Hầu hết các phương tiện nhập liệu mềm đều cung cấp một button action cho người dùng ở góc dưới cùng, phù hợp với trường văn bản hiện tại. Theo mặc định, hệ thống sử dụng button này cho cả action **Next (tiếp theo)** hoặc **Done (hoàn thành)**, trừ phi trường văn bản của bạn cho phép có nhiều dòng (chẳng hạn như với `android:inputType="textMultiLine"`); trong trường hợp này, button action là dấu ngắt dòng. Tuy nhiên, bạn có thể xác định thêm những action có khả năng phù hợp với trường văn bản hơn, chẳng hạn như **Send** hay **Go**.

Để xác định button action cho bàn phím, hãy sử dụng thuộc tính `android:imeOptions` với một giá trị action như `"actionSend"` hay `"actionSearch"`. Ví dụ:



Hình 4. Button Send xuất hiện khi bạn khai báo `android:imeOptions="actionSend"`.

```
<EditText
    android:id="@+id/search"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:hint="@string/search_hint"
    android:inputType="text"
    android:imeOptions="actionSend" />
```

Sau đó, bạn có thể lắng nghe phím được nhấn trên button action bằng cách xác định một [TextView.OnEditorActionListener](#) cho phần tử [EditText](#). Trong trình

lắng nghe, hãy phản hồi ID của action IME (input method editor - trình chỉnh sửa phương tiện nhập liệu) phù hợp xác định trong lớp [EditorInfo](#), chẳng hạn như [IME_ACTION_SEND](#). Ví dụ:

```
EditText editText = (EditText) findViewById(R.id.search);
editText.setOnEditorActionListener(new OnEditorActionListener() {
    @Override
    public boolean onEditorAction(TextView v, int actionId, KeyEvent event) {
        boolean handled = false;
        if (actionId == EditorInfo.IME_ACTION_SEND) {
            sendMessage();
            handled = true;
        }
        return handled;
    }
});
```

6.2 Xử lý trạng thái hiển thị của phương tiện nhập liệu

Khi một trường văn bản nhận hoặc bị mất focus Android hiển thị hoặc ẩn phương tiện nhập liệu (chẳng hạn như bàn phím trên màn hình) một cách tương ứng. Hệ thống cũng ra quyết định về cách thức giao diện người dùng và trường văn bản xuất hiện trên phương tiện nhập liệu. Ví dụ, khi không gian dọc trên màn hình bị ràng buộc, trường văn bản sẽ lấp đầy toàn bộ không gian trên phương tiện nhập liệu. Đối với hầu hết ứng dụng, các hành vi mặc định đã thỏa mãn nhu cầu.

Dù vậy, trong một số trường hợp, bạn có thể muốn điều khiển trạng thái hiển thị được của phương tiện nhập liệu trực tiếp hơn và xác định cách bạn muốn layout xuất hiện khi phương tiện nhập liệu hiện diện. Mục này sẽ giải thích cách điều khiển và phản hồi trạng thái hiển thị được của phương tiện nhập liệu.

Hiển thị phương tiện nhập liệu khi activity khởi động

Mặc dù đặt focus vào trường văn bản thứ nhất trong layout khi activity khởi động, song Android lại không hiển thị phương tiện nhập liệu. Hành vi này là phù hợp, bởi việc nhập văn bản có thể không phải là nhiệm vụ chính trong activity đó. Tuy nhiên, nếu việc nhập văn bản là nhiệm vụ chính (chẳng hạn như ở màn hình đăng nhập), bạn sẽ muốn phương tiện nhập liệu xuất hiện theo mặc định.

Để hiển thị phương tiện nhập liệu khi activity khởi động, hãy thêm thuộc tính `android:windowSoftInputMode` cho phần tử `<activity>` với giá trị `"stateVisible"`. Ví dụ:

```
<application ... >
    <activity
        android:windowSoftInputMode="stateVisible" ... >
        ...
    </activity>
    ...
</application>
```

Ghi chú: Nếu thiết bị của người dùng có bàn phím cứng đi kèm, phương tiện nhập liệu mềm sẽ *không* xuất hiện.

Hiển thị phương tiện nhập liệu theo nhu cầu

Nếu có một phương tiện nhập liệu trong vòng đời của activity mà tại đây, bạn muốn đảm bảo phương tiện nhập liệu phải hiện diện, hãy sử dụng [InputMethodManager](#) để hiển thị nó.

Ví dụ, phương thức sau lấy một [View](#) mà tại đây, người dùng cần phải nhập liệu vào View này, gọi [requestFocus\(\)](#) để focus vào View, sau đó gọi [showSoftInput\(\)](#) để mở phương tiện nhập liệu:

```
public void showSoftKeyboard(View view) {
    if (view.requestFocus()) {
        InputMethodManager imm = (InputMethodManager)
            getSystemService(Context.INPUT_METHOD_SERVICE);
        imm.showSoftInput(view, InputMethodManager.SHOW_IMPLICIT);
    }
}
```

Ghi chú: Một khi phương tiện nhập liệu đã hiển thị, bạn không nên ẩn nó bằng cách lập trình mã nguồn nữa. Hệ thống ẩn phương tiện nhập liệu khi người dùng hoàn thành tác vụ trong trường văn bản, hoặc người dùng có thể ẩn nó với một điều khiển của hệ thống (chẳng hạn như với button *Back*).

Xác định cách thức phản hồi của giao diện người dùng

Khi phương tiện nhập liệu xuất hiện trên màn hình, nó sẽ làm giảm không gian hiện có của giao diện người dùng trong ứng dụng. Hệ thống ra quyết định là nó nên điều chỉnh phần hiển thị của giao diện người dùng, nhưng có thể nó không thực hiện điều này ngay lúc đó. Để đảm bảo hành vi tốt nhất cho ứng dụng của mình, hãy chỉ ra cách bạn muốn hệ thống hiển thị giao diện người dùng của mình trong phần không gian còn lại.

Để khai báo cách phản hồi mà bạn mong muốn trong một activity, sử dụng thuộc tính `android:windowSoftInputMode` trong phần tử `<activity>` của file kê khai với một trong số những giá trị “adjust” (“điều chỉnh”).

Lập trình Android cơ bản

Ví dụ, nhằm đảm bảo hệ thống thay đổi kích thước layout cho phần không gian hiện tại - điều này đảm bảo rằng mọi nội dung của layout đều có thể được truy cập - sử dụng "adjustResize":

```
<application ... >
    <activity
        android:windowSoftInputMode="adjustResize" ... >
        ...
    </activity>
    ...
</application>
```

Bạn có thể kết hợp thông số điều chỉnh ở trên với thông số về [trạng thái hiển thị của phương thức nhập liệu ban đầu](#) từ bên trên:

```
<activity
    android:windowSoftInputMode="stateVisible|adjustResize" ... >
    ...
</activity>
```

Xác định "adjustResize" là việc thực sự quan trọng nếu giao diện người dùng của bạn bao gồm các điều khiển mà người dùng có thể cần để truy cập ngay lập tức sau khi hoặc trong khi thực hiện nhập văn bản đầu vào. Ví dụ, nếu bạn sử dụng một layout tương đối để đặt một button bar ở cuối màn hình, hãy dùng "adjustResize" để thay đổi kích thước layout sao cho bar đó xuất hiện trên phương tiện nhập liệu.

6.3 Hỗ trợ điều hướng qua bàn phím

Ngoài các phương tiện nhập liệu mềm (chẳng hạn như bàn phím trên màn hình), Android còn hỗ trợ các bàn phím vật lý gắn vào thiết bị. Bàn phím không chỉ đem lại một chế độ thuận tiện cho việc nhập liệu văn bản mà còn cung cấp một cách thức để người dùng điều hướng và tương tác với ứng dụng. Mặc dù hầu hết thiết bị cầm tay như điện thoại đều sử dụng cảm ứng làm chế độ tương tác chính, song máy tính bảng và các thiết bị tương tự ngày càng trở nên phổ biến và nhiều người dùng thích sử dụng phụ kiện bàn phím gắn thêm vào.

Như nhiều thiết bị Android từng cung cấp loại trải nghiệm này, điều quan trọng là bạn phải tối ưu ứng dụng của mình để hỗ trợ tương tác qua bàn phím. Mục này mô tả cách bạn có thể hỗ trợ tốt hơn cho việc điều hướng với bàn phím.

Ghi chú: Việc hỗ trợ điều hướng theo hướng trong ứng dụng của bạn rất quan trọng để đảm bảo người dùng có thể [truy cập được](#) mà không cần dùng các hỗ trợ trực quan. Việc hỗ trợ đầy đủ việc điều hướng theo hướng trong ứng dụng cũng giúp bạn [kiểm thử giao diện người dùng](#) với các công cụ như [uiautomator](#).

Kiểm thử ứng dụng

Người dùng có thể đã duyệt ứng dụng của bạn bằng bàn phím, bởi vì hệ thống Android bật hầu hết các hành vi cần thiết theo mặc định.

Tất cả các widget tương tác cung cấp bởi framework Android (chẳng hạn như [Button](#) và [EditText](#)) đều có thể nhận focus. Điều này có nghĩa rằng, người dùng có thể điều hướng với các thiết bị điều khiển như D-pad hoặc bàn phím và mỗi widget sẽ sáng lên hoặc ngược lại, thay đổi giao diện của nó khi nó được focus vào.

Để kiểm thử ứng dụng của bạn, hãy:

1. Cài đặt ứng dụng trên một thiết bị có sẵn bàn phím phần cứng.

Nếu thiết bị phần cứng của bạn không có bàn phím, hãy kết nối bàn phím Bluetooth hoặc một bàn phím USB (mặc dù không phải mọi thiết bị đều hỗ trợ phụ kiện USB).

Bạn cũng có thể sử dụng trình giả lập (emulator) Android:

1. Trong AVD Manager, nhấn vào **New Device** (thiết bị mới) hoặc chọn một profile có sẵn và nhấn **Clone** (tạo bản sao).
2. Trong cửa sổ hiện ra, hãy đảm bảo rằng **Keyboard** và **DPad** được bật.
2. Để kiểm thử ứng dụng, hãy chỉ dùng phím Tab để điều hướng trên giao diện người dùng, đảm bảo rằng mỗi điều khiển trong đó đều nhận được focus khi cần.

Tìm xem có thực thể nào được focus không như ý bạn muốn không.

3. Bắt đầu từ khi ứng dụng khởi động và thay vào đó, hãy dùng các phím điều hướng (phím mũi tên trên bàn phím) để duyệt ứng dụng.

Từ mỗi phần tử có thể được focus trong giao diện người dùng, nhấn Up (lên), Down (xuống), Left (trái), Right (phải).

Tìm xem có thực thể nào được focus không như ý bạn muốn không.

Nếu bạn gặp bất cứ thực thể nào mà tại đó khi bạn đang điều hướng với phím Tab hoặc điều khiển điều hướng không như ý muốn, hãy xác định vị trí nên được focus trong layout. Điều này sẽ được bàn luận ở mục sau.

Xử lý việc điều hướng dùng phím Tab

Khi người dùng điều hướng trên ứng dụng của bạn bằng phím Tab, hệ thống chuyển focus giữa các phần tử dựa trên thứ tự mà chúng xuất hiện trong layout. Ví dụ, nếu bạn sử dụng một layout tương đối (relative layout) và thứ tự các phần tử trên màn hình khác với trong file, bạn phải tự xác định thứ tự focus theo cách thủ công.

Ví dụ, trong layout dưới đây, hai button được sắp xếp ở bên phải và một trường văn bản được sắp xếp phía bên trái của button thứ hai. Để truyền focus từ button thứ nhất tới trường văn bản, sau đó là button thứ hai, layout phải xác định rõ thứ tự focus cho từng phần tử mà có thể focus vào với thuộc tính `android:nextFocusForward`:


```
<RelativeLayout ...>
    <Button
        android:id="@+id/button1"
        android:layout_alignParentTop="true"
        android:layout_alignParentRight="true"
        android:nextFocusForward="@+id/editText1"
        ... />
    <Button
        android:id="@+id/button2"
        android:layout_below="@id/button1"
        android:nextFocusForward="@+id/button1"
        ... />
    <EditText
        android:id="@id/editText1"
        android:layout_alignBottom="@+id/button2"
        android:layout_toLeftOf="@id/button2"
        android:nextFocusForward="@+id/button2"
        ... />
    ...
</RelativeLayout>
```

Lúc này, thay vì chuyển focus từ `button1` tới `button2`, sau đó là `editText1`, việc chuyển focus phù hợp với giao diện trên màn hình: Từ `button1` tới `editText1` rồi tới `button2`.

Xử lý việc điều hướng theo hướng

Người dùng cũng có thể điều hướng ứng dụng của bạn bằng cách dùng các phím mũi tên trên bàn phím (hành vi tương tự như khi điều hướng với D-pad hoặc bi lăn). Hệ thống cung cấp một dự đoán tốt nhất cho view nên được focus, theo hướng đã xác định dựa trên layout của các view trên màn hình. Đôi khi, hệ thống có thể đoán sai.

Nếu hệ thống không truyền focus cho view thích hợp khi người dùng thao tác điều hướng theo hướng đã cho, hãy xác định view nào nên được focus với các thuộc tính sau:

- `android:nextFocusUp`.
- `android:nextFocusDown`.
- `android:nextFocusLeft`.
- `android:nextFocusRight`.

Mỗi thuộc tính chỉ định view kế tiếp nhận focus khi người dùng thực hiện điều hướng theo hướng bằng cách sử dụng ID của view. Ví dụ:

```

<Button
    android:id="@+id/button1"
    android:nextFocusRight="@+id/button2"
    android:nextFocusDown="@+id/editText1"
    ... />
<Button
    android:id="@+id/button2"
    android:nextFocusLeft="@+id/button1"
    android:nextFocusDown="@+id/editText1"
    ... />
<EditText
    android:id="@+id/editText1"
    android:nextFocusUp="@+id/button1"
    ... />

```

6.4 Xử lý các action từ bàn phím

Khi người dùng focus vào một view có văn bản chỉnh sửa được, chẳng hạn như một phần tử [EditText](#) và người dùng có một bàn phím cứng gắn vào, tất cả đầu vào sẽ được hệ thống xử lý. Tuy nhiên, nếu muốn chặn hoặc tự xử lý trực tiếp đầu vào từ bàn phím, bạn có thể thực thi các phương thức callback từ giao diện [KeyEvent.Callback](#), chẳng hạn như [onKeyDown\(\)](#) và [onKeyMultiple\(\)](#).

Cả hai lớp [Activity](#) và [View](#) đều thực thi giao diện [KeyEvent.Callback](#); vì vậy, thường thì bạn nên ghi đè các phương thức callback trong phần mở rộng của những lớp đó, nếu thích hợp.

Ghi chú: Khi xử lý các sự kiện từ bàn phím với lớp [KeyEvent](#) và những hàm API liên quan, bạn nên hy vọng rằng các sự kiện ấy sẽ chỉ tới từ bàn phím phần cứng. Đừng bao giờ trông cậy vào các sự kiện từ bất cứ phím ảo nào trên phương tiện nhập liệu mềm (một bàn phím trên màn hình chẳng hạn).

Xử lý các sự kiện phím đơn

Để xử lý một phím đơn được nhấn, hãy thực thi [onKeyDown\(\)](#) hoặc [onKeyUp\(\)](#) nếu thích hợp. Thường thì bạn nên sử dụng [onKeyUp\(\)](#) nếu muốn đảm bảo chỉ nhận một sự kiện. Nếu người dùng nhấn và giữ button, [onKeyDown\(\)](#) sẽ được gọi nhiều lần.

Ví dụ, phương thức sau sẽ phản hồi một số phím để điều khiển một trò chơi:

```

@Override
public boolean onKeyUp(int keyCode, KeyEvent event) {
    switch (keyCode) {
        case KeyEvent.KEYCODE_D:
            moveShip(MOVE_LEFT);
            return true;
        case KeyEvent.KEYCODE_F:

```